

Atividade Prática 1 – parte 1 – APPOO

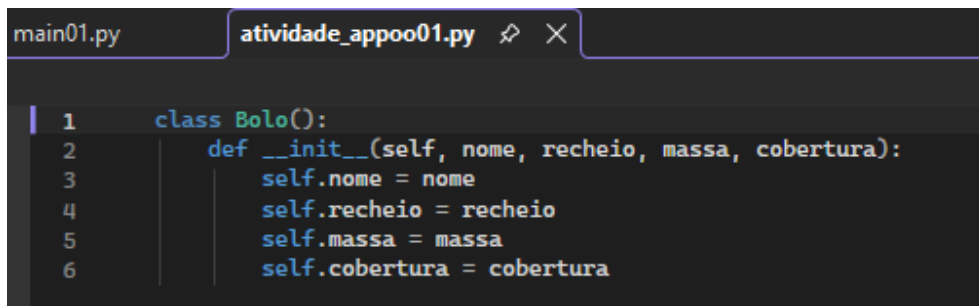
Nome: Guilherme Augusto Nunes

Matrícula: 2024017015

Exercício 1

Criação da instância da classe *Bolo* e adição seus atributos:

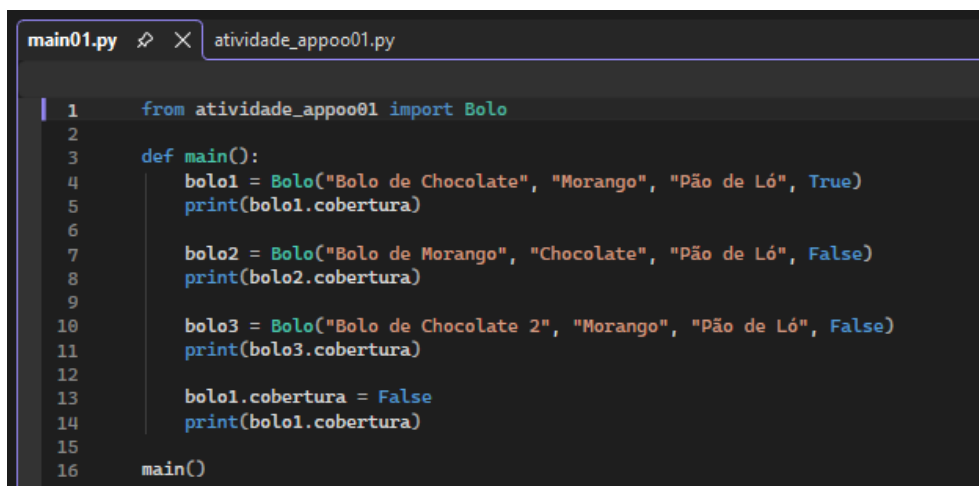
- *nome* como string;
- *recheio* como string;
- *massa* como string;
- *cobertura* como bool (apenas retorna true se tem cobertura).



```
main01.py  atividade_appoo01.py  ✖  ✕  
  
1  class Bolo():  
2      def __init__(self, nome, recheio, massa, cobertura):  
3          self.nome = nome  
4          self.recheio = recheio  
5          self.massa = massa  
6          self.cobertura = cobertura
```

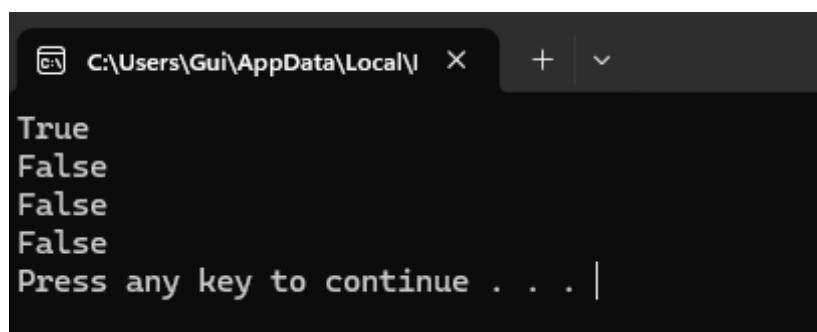
Figura 1 Módulo com classe *Bolo*

Criei os três primeiros objetos da classe *Bolo* (bolo1, bolo2 e bolo3) e logo depois eu alterei diretamente o valor do atributo *cobertura* do objeto *Bolo1* agora ele passa a não ter mais cobertura — o esperado é que na quarta linha seja impresso “false”.



```
main01.py  ✖  ✕  atividade_appoo01.py  
  
1  from atividade_appoo01 import Bolo  
2  
3  def main():  
4      bolo1 = Bolo("Bolo de Chocolate", "Morango", "Pão de Ló", True)  
5      print(bolo1.cobertura)  
6  
7      bolo2 = Bolo("Bolo de Morango", "Chocolate", "Pão de Ló", False)  
8      print(bolo2.cobertura)  
9  
10     bolo3 = Bolo("Bolo de Chocolate 2", "Morango", "Pão de Ló", False)  
11     print(bolo3.cobertura)  
12  
13     bolo1.cobertura = False  
14     print(bolo1.cobertura)  
15  
16     main()
```

Figura 2 Main com testes e objetos criados



```
C:\Users\Gui\AppData\Local\I  ✖  +  ▾  
  
True  
False  
False  
False  
Press any key to continue . . . |
```

Figura 3 Saída do código no terminal

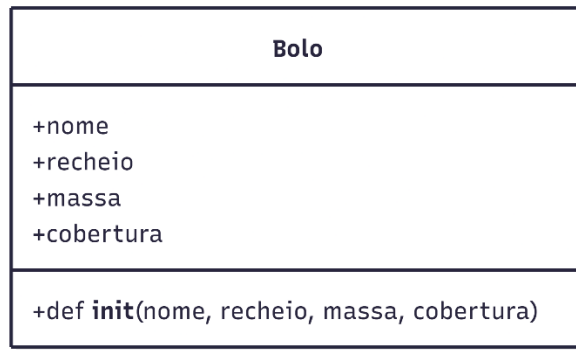


Figura 4 Diagrama UML

Exercício 2

Criação da instância da classe *FazBolo* e adição seus atributos e método *assar*:

```

main02.py  atividade_apoo02.py  x
FazBolo  tempo
1  class FazBolo():
2      def __init__(self, massa, recheio, cobertura):
3          self.massa = massa
4          self.recheio = recheio
5          self.cobertura = cobertura
6
7      def assar(self, temperatura, tempo):
8          if temperatura not in ["alta", "media", "baixa"]:
9              print("Temperatura inválida. Por favor, escolha entre 'alta', 'media' ou 'baixa'.")
10             return
11          if tempo <= 0 or tempo > 180:
12              print("Tempo inválido. Por favor, insira um tempo positivo e menor ou igual a 180.")
13              return
14          else:
15              self.temperatura = temperatura
16              self.tempo = tempo
17
18          print(
19              f"Você está querendo assar o bolo de massa com recheio de {self.recheio} e cobertura de {self.cobertura} no fogo temperatura por "
20              f"{tempo} minutos", sendo {tempo} e {temperatura} inseridos pelo usuário"
21          )
22
23          if temperatura == "alta":
24              print("Para o fogo alto, o bolo assa em 10 minutos. Antes fica cru e depois ele queima.")
25
26          elif temperatura == "media":
27              print("Para o fogo médio, o bolo assa em 30 minutos. Antes fica cru e depois ele queima.")
28
29          elif temperatura == "baixa":
30              print("Para o fogo baixo, o bolo assa em 45 minutos. Antes fica cru e depois ele queima.")

```

Figura 5 Módulo com classe *FazBolo*, instância e método *assar*

Coloquei as condicionais necessárias para que os valores não sejam negativos ou ultrapassem 180.

Main com os testes realizados com valores absurdos e negativos:

```
main02.py  atividade_appoo02.py*
1  from atividade_appoo02 import FazBolo
2  def main():
3      Bolo1 = FazBolo("Pão de Ló", "Morango", "Chocolate")
4      Bolo1.assar("media", 90)
5
6      print("\n")
7
8      Bolo2 = FazBolo("Chocolate", "Doce de Leite", "Brigadeiro")
9      Bolo2.assar("alta", 100000)
10
11     print("\n")
12
13     Bolo3 = FazBolo("Baunilha", "Creme", "Frutas")
14     Bolo3.assar("baixa", -45)
15
16     main()
```

Figura 6 Main com testes e objetos criados

```
C:\Users\Gui\AppData\Local\  +  -  x
Você está querendo assar o bolo de massa com recheio de Morango e cobertura de Chocolate no fogo temperatura por 90 minutos, sendo 90 e media inseridos pelo usuário
Para o fogo médio, o bolo assa em 30 minutos. Antes fica cru e depois ele queima.

Tempo inválido. Por favor, insira um tempo positivo e menor ou igual a 120.

Tempo inválido. Por favor, insira um tempo positivo e menor ou igual a 120.
Press any key to continue . . . |
```

Figura 7 Saída no terminal

Funcionou corretamente, para valores negativos e absurdos ele exibe a mensagem de tempo inválido e para valores correto, ele exibe a mensagem esperada.

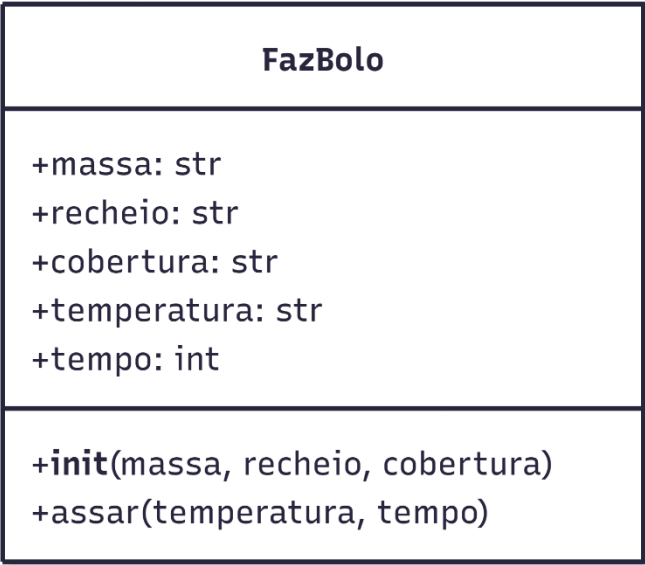
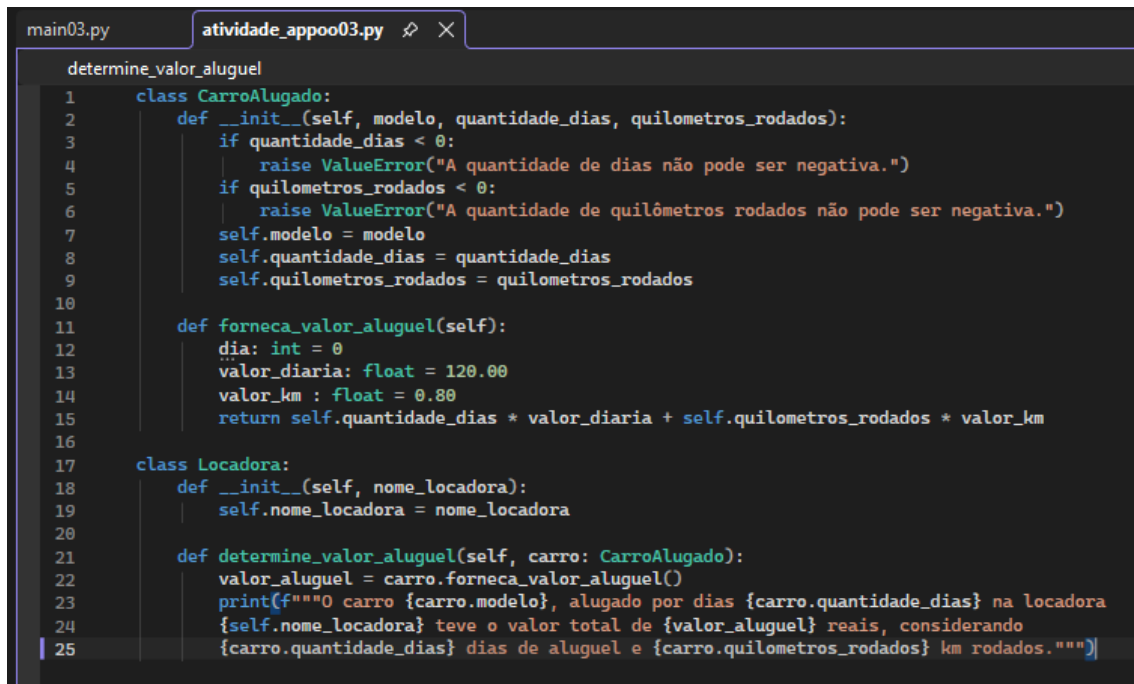


Figura 8 Diagrama UML

Exercício 3

Criação da instância da classe *CarroAlugado* e *Locadora* e adição seus respectivos atributos e métodos:



```
main03.py  atividade_app003.py  ✖  ×

determine_valor_aluguel
1  class CarroAlugado:
2      def __init__(self, modelo, quantidade_dias, quilometros_rodados):
3          if quantidade_dias < 0:
4              raise ValueError("A quantidade de dias não pode ser negativa.")
5          if quilometros_rodados < 0:
6              raise ValueError("A quantidade de quilômetros rodados não pode ser negativa.")
7          self.modelo = modelo
8          self.quantidade_dias = quantidade_dias
9          self.kilometros_rodados = quilometros_rodados
10
11     def forneca_valor_aluguel(self):
12         dia: int = 0
13         valor_diaria: float = 120.00
14         valor_km : float = 0.80
15         return self.quantidade_dias * valor_diaria + self.kilometros_rodados * valor_km
16
17     class Locadora:
18         def __init__(self, nome_locadora):
19             self.nome_locadora = nome_locadora
20
21         def determine_valor_aluguel(self, carro: CarroAlugado):
22             valor_aluguel = carro.forneca_valor_aluguel()
23             print(f"""O carro {carro.modelo}, alugado por dias {carro.quantidade_dias} na locadora
24                 {self.nome_locadora} teve o valor total de {valor_aluguel} reais, considerando
25                 {carro.quantidade_dias} dias de aluguel e {carro.kilometros_rodados} km rodados.""")
```

Figura 9 Módulo com classes, instâncias e métodos

Aqui há a troca de mensagens entre objetos, quando `valor_aluguel = carro.forneca_valor_aluguel()`, pois um método da classe *Locadora* solicitando a um método da classe *CarroAlugado*.

Para o aviso de erro de valor, usei a função `raise ValueError()`, que lança uma exceção quando um valor é negativo.

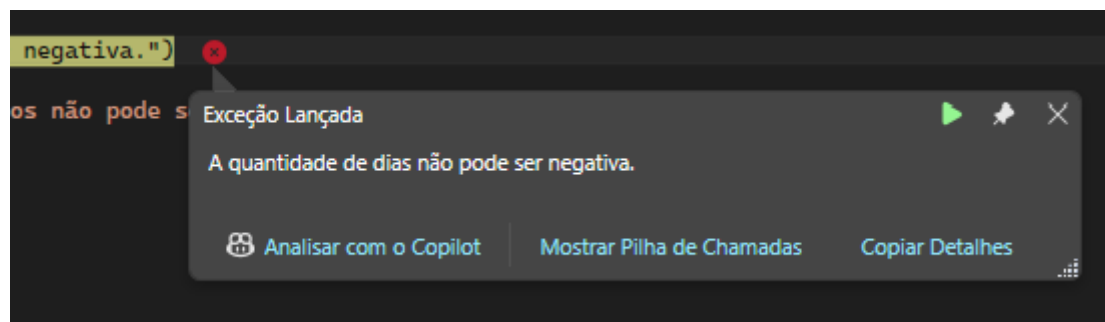


Figura 10 Exceção lançada quando dia é um valor negativo

Main com testes corretos:

```
main03.py atividade_appoo03.py
1 from atividade_appoo03 import CarroAlugado, Locadora
2
3 def main():
4     locadora1 = Locadora("Locadora A")
5     carro1 = CarroAlugado("Fusion", 5, 300)
6     locadora1.determine_valor_aluguel(carro1)
7
8     locadora2 = Locadora("Locadora B")
9     carro2 = CarroAlugado("Civic", 3, 150)
10    locadora2.determine_valor_aluguel(carro2)
11    main()
```

Figura 11 Main com objetos criados e testes feitos

Em `locadora1.determine_valor_aluguel(carro1)` e `locadora2.determine_valor_aluguel(carro2)` também há troca de mensagens entre os objetos.

```
C:\Users\Gui\AppData\Local\I + v
0 carro Fusion, alugado por dias 5 na locadora
  Locadora A teve o valor total de 840.0 reais, considerando
  5 dias de aluguel e 300 km rodados.
0 carro Civic, alugado por dias 3 na locadora
  Locadora B teve o valor total de 480.0 reais, considerando
  3 dias de aluguel e 150 km rodados.
Press any key to continue . . . |
```

Figura 12 Saída no terminal

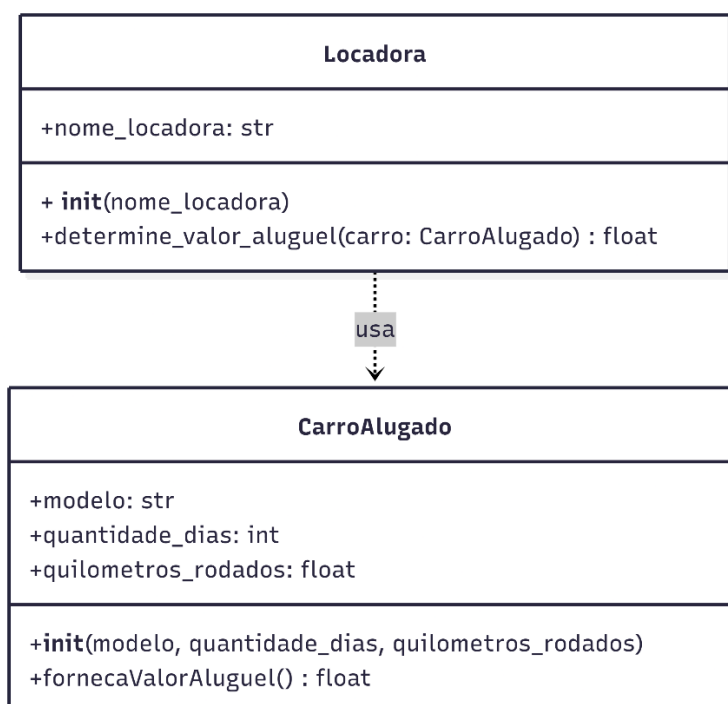


Figura 13 Diagrama UML

Exercício 4

Criação da instância da classe *Pedido* e *Cafeteira* e adição seus respectivos atributos e métodos:

```
main04.py | atividade_appoo04.py | Pedido
1 class Pedido:
2     def __init__(self, tipo_cafe, quantidade):
3         self.tipo_cafe = tipo_cafe
4         self.quantidade = quantidade
5
6 class Cafeteira:
7     def __init__(self, modelo_maquina):
8         self.modelo_maquina = modelo_maquina
9
10    def preparar_cafe(self, pedido):
11        tempo_preparo: int = 2
12        if pedido.quantidade < 1:
13            raise ValueError("Quantidade inválida. O pedido deve conter pelo menos 1 café.")
14        return 0
15        tempo_total: int = tempo_preparo * pedido.quantidade
16        print(f"Preparando {pedido.quantidade} x {pedido.tipo_cafe} na máquina {self.modelo_maquina}. Tempo estimado: {tempo_total} minutos.")
17        return tempo_total
```

Figura 14 Módulo com classes, instâncias e métodos

Novamente com a função `raise ValueError()` para lançamento de exceções.

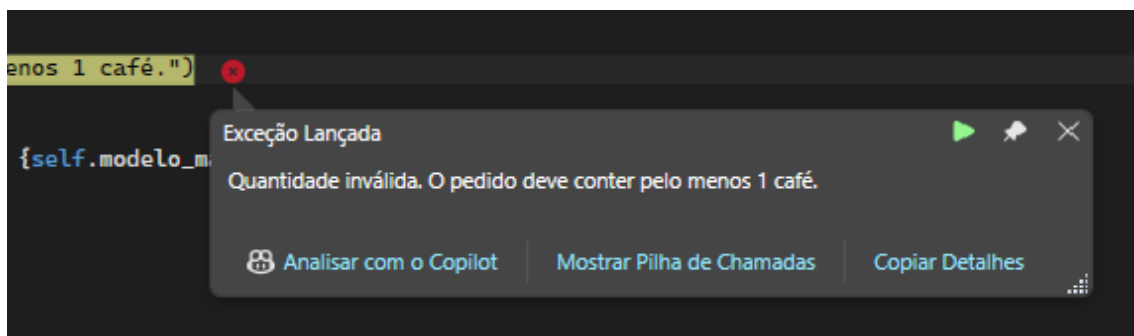


Figura 15 Exceção lançada quando pedido é um valor negativo ou 0

```
main04.py* | atividade_appoo04.py
main
1 from atividade_appoo04 import *
2
3 def main():
4     pedido1 = Pedido("Café Espresso", 2)
5     cafeteira1 = Cafeteira("Nespresso 1")
6     cafeteira1.preparar_cafe(pedido1)
7
8     pedido2 = Pedido("Café Espresso", 5)
9     cafeteira2 = Cafeteira("Nespresso 2")
10    cafeteira2.preparar_cafe(pedido2)
11
12    pedido3 = Pedido("Capuccino", 4)
13    cafeteira3 = Cafeteira("Nespresso")
14    cafeteira3.preparar_cafe(pedido3)
15
16    main()
```

Figura 16 Main com objetos criados e testes feitos

Aqui há troca de mensagens entre objetos, *pedido1* é um objeto de *Pedido* e *preparar_cafe()* é um método da classe *Cafeteira*.

```
C:\Users\Gui\AppData\Local\l X + v
Preparando 2 x Café Expresso na máquina Nespresso 1. Tempo estimado: 4 minutos.
Preparando 5 x Café Expresso na máquina Nespresso 2. Tempo estimado: 10 minutos.
Preparando 4 x Capuccino na máquina Nespresso. Tempo estimado: 8 minutos.
Press any key to continue . . . |
```

Figura 17 Saída no terminal

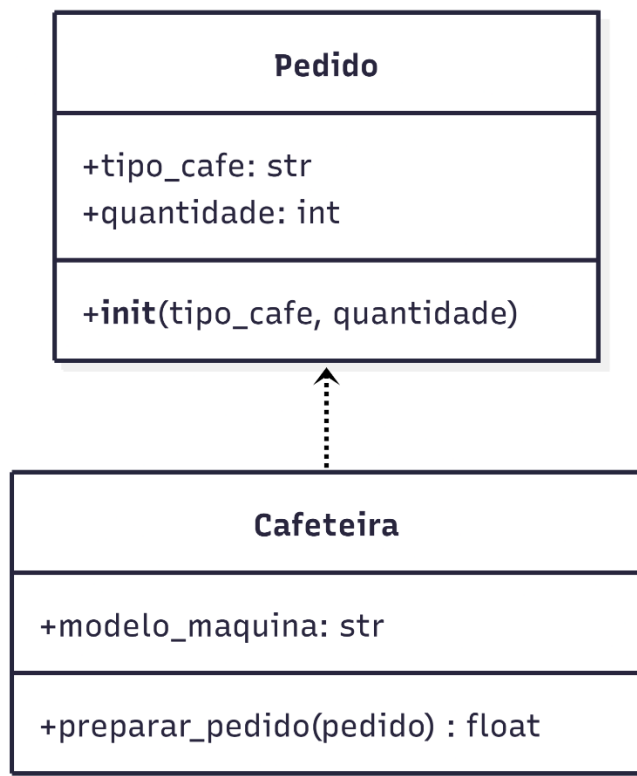


Figura 18 Diagrama UML

Atividade Prática 1 – parte 2 – APPOO

Exercício 1

Criação do módulo principal contendo as classes e subclasses pedidas pelo enunciado.

```
1 class Veiculo:
2     def __init__(self, marca: str, modelo: str, ano_fabricacao: int):
3         if not marca.strip():
4             raise TypeError ("A marca não pode ser vazia.")
5         self.marca = marca
6
7         if not modelo.strip():
8             raise TypeError ("O modelo não pode ser vazio.")
9         self.modelo = modelo
10
11         if ano_fabricacao not in range (1886, 2025):
12             raise TypeError ("Digite um ano válido")
13         self.ano_fabricacao = ano_fabricacao
14
15     def exibir_info(self):
16         print(f"Veiculo: {self.marca} {self.modelo} - Ano: {self.ano_fabricacao}")
17
18 class Carro(Veiculo):
19     def __init__(self, marca: str, modelo: str, ano_fabricacao: int, numero_portas: int):
20         super().__init__(marca, modelo, ano_fabricacao)
21         if numero_portas not in [2, 4]:
22             raise ValueError ("A quantidade de portas deve ser 2 ou 4")
23         self.numero_portas = numero_portas
24
25     def exibir_info(self):
26         super().exibir_info()
27         print(f"Número de portas: {self.numero_portas}")
28
29 class Moto(Veiculo):
30     def __init__(self, marca: str, modelo: str, ano_fabricacao: int, cilindrada: int):
31         super().__init__(marca, modelo, ano_fabricacao)
32         if cilindrada <= 0:
33             raise ValueError ("A cilindrada deve ser maior que 0")
34         self.cilindrada = cilindrada
35
36     def exibir_info(self):
37         super().exibir_info()
38         print(f"Cilindrada: {self.cilindrada}")
39
40 class Caminhao(Veiculo):
41     def __init__(self, marca: str, modelo: str, ano_fabricacao: int, capacidade_carga_toneladas: int):
42         super().__init__(marca, modelo, ano_fabricacao)
43         if capacidade_carga_toneladas <= 0:
44             raise ValueError ("A capacidade de carga deve ser maior que 0")
45         self.capacidade_carga_toneladas = capacidade_carga_toneladas
46
47     def exibir_info(self):
48         super().exibir_info()
49         print(f"Capacidade de carga: {self.capacidade_carga_toneladas} toneladas")
```

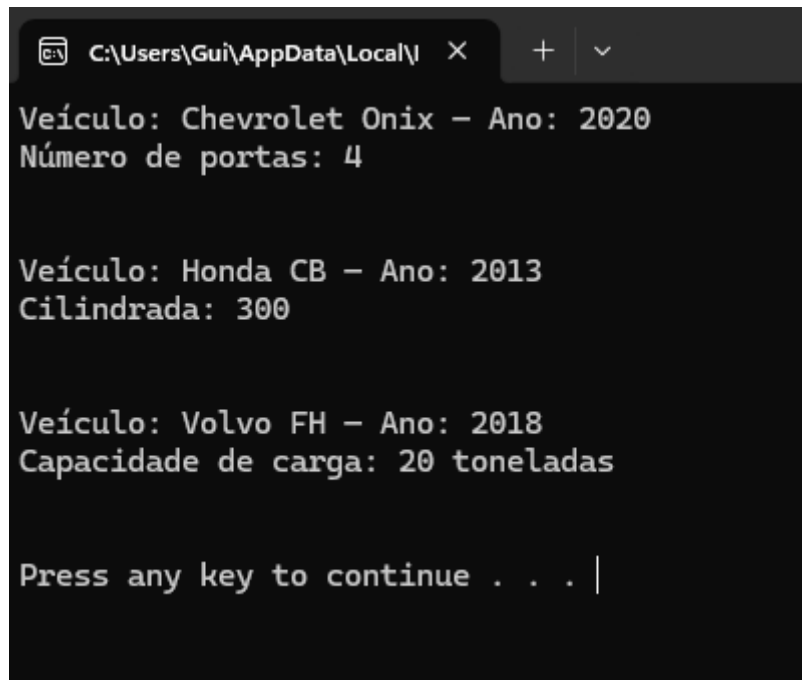
Figura 19 Módulo com classes e métodos

Main com os objetos instanciados corretamente e objetos instanciados com erros propositalis.

```
1  from pratica02_01 import *
2
3  def main():
4      carro1 = Carro("Chevrolet", "Onix", 2020, 4)
5      carro1.exibir_info()
6      print("\n")
7
8      moto1 = Moto("Honda", "CB", 2013, 300)
9      moto1.exibir_info()
10     print("\n")
11
12     caminhao1 = Caminhao("Volvo", "FH", 2018, 20)
13     caminhao1.exibir_info()
14     print("\n")
15
16     print("\nTestes Incorretos:")
17
18     carro2 = Carro("Fiat", "Uno", 1800, 4)
19     carro2.exibir_info()
20     print("\n")
21
22     moto2 = Moto("Yamaha", "MT-07", 2022, -700)
23     moto2.exibir_info()
24     print("\n")
25
26     caminhao2 = Caminhao("Scania", "R500", 2019, 0)
27     caminhao2.exibir_info()
28     print("\n")
29
30     carro3 = Carro("Ford", "Ka", 2021, 3)
31     carro3.exibir_info()
32     print("\n")
33
34     carro4 = Carro("", "Ka", 2021, 4)
35     carro4.exibir_info()
36     print("\n")
37
38     main()
```

Figura 20 Main

Saída com os objetos corretamente instanciados



```
C:\Users\Gui\AppData\Local\I X + v

Veículo: Chevrolet Onix – Ano: 2020
Número de portas: 4

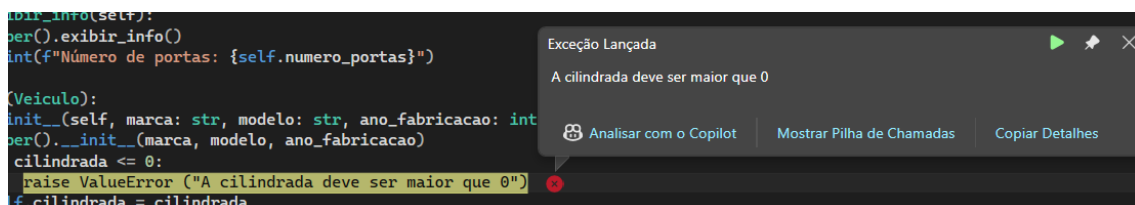
Veículo: Honda CB – Ano: 2013
Cilindrada: 300

Veículo: Volvo FH – Ano: 2018
Capacidade de carga: 20 toneladas

Press any key to continue . . . |
```

Figura 21 Saída no terminal

Tratamento de Exceções:



```
def __init__(self, marca: str, modelo: str, ano_fabricacao: int, cilindrada: int):
    super().__init__(marca, modelo, ano_fabricacao)
    self.cilindrada = cilindrada

    if cilindrada <= 0:
        raise ValueError("A cilindrada deve ser maior que 0")

def exibir_info(self):
    print(f"Número de portas: {self.numero_portas}")

class Veiculo:
    def __init__(self, marca: str, modelo: str, ano_fabricacao: int, cilindrada: int):
        super().__init__(marca, modelo, ano_fabricacao)
        self.cilindrada = cilindrada

    def exibir_info(self):
        print(f"Número de portas: {self.numero_portas}")

    def __init__(self, marca: str, modelo: str, ano_fabricacao: int, cilindrada: int):
        super().__init__(marca, modelo, ano_fabricacao)
        self.cilindrada = cilindrada

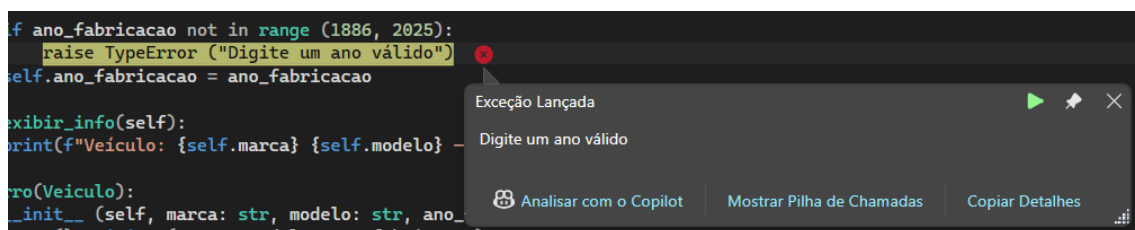
    if cilindrada <= 0:
        raise ValueError("A cilindrada deve ser maior que 0")

    def exibir_info(self):
        print(f"Número de portas: {self.numero_portas}")
```

Exceção Lançada
A cilindrada deve ser maior que 0

Analisar com o Copilot | Mostrar Pilha de Chamadas | Copiar Detalhes

Figura 22 Exceção lançada (erro de cilindrada)



```
if ano_fabricacao not in range(1886, 2025):
    raise TypeError("Digite um ano válido")
self.ano_fabricacao = ano_fabricacao

def exibir_info(self):
    print(f"Veículo: {self.marca} {self.modelo} – Ano: {self.ano_fabricacao}")

class Veiculo:
    def __init__(self, marca: str, modelo: str, ano_fabricacao: int, capacidade_carga_toneladas: int):
        super().__init__(marca, modelo, ano_fabricacao)
        self.ano_fabricacao = ano_fabricacao
        self.capacidade_carga_toneladas = capacidade_carga_toneladas

    def exibir_info(self):
        print(f"Veículo: {self.marca} {self.modelo} – Ano: {self.ano_fabricacao}")

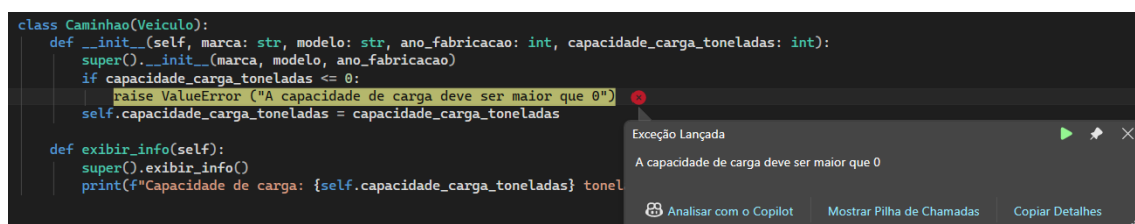
    def __init__(self, marca: str, modelo: str, ano_fabricacao: int, capacidade_carga_toneladas: int):
        super().__init__(marca, modelo, ano_fabricacao)
        self.ano_fabricacao = ano_fabricacao
        self.capacidade_carga_toneladas = capacidade_carga_toneladas

    def exibir_info(self):
        print(f"Veículo: {self.marca} {self.modelo} – Ano: {self.ano_fabricacao}")
```

Exceção Lançada
Digite um ano válido

Analisar com o Copilot | Mostrar Pilha de Chamadas | Copiar Detalhes

Figura 23 Exceção lançada (erro de ano inválido)



```
class Caminhao(Veiculo):
    def __init__(self, marca: str, modelo: str, ano_fabricacao: int, capacidade_carga_toneladas: int):
        super().__init__(marca, modelo, ano_fabricacao)
        self.capacidade_carga_toneladas = capacidade_carga_toneladas

    def exibir_info(self):
        super().exibir_info()
        print(f"Capacidade de carga: {self.capacidade_carga_toneladas} toneladas")

class Veiculo:
    def __init__(self, marca: str, modelo: str, ano_fabricacao: int, capacidade_carga_toneladas: int):
        super().__init__(marca, modelo, ano_fabricacao)
        self.ano_fabricacao = ano_fabricacao
        self.capacidade_carga_toneladas = capacidade_carga_toneladas

    def exibir_info(self):
        print(f"Veículo: {self.marca} {self.modelo} – Ano: {self.ano_fabricacao}")

    def __init__(self, marca: str, modelo: str, ano_fabricacao: int, capacidade_carga_toneladas: int):
        super().__init__(marca, modelo, ano_fabricacao)
        self.ano_fabricacao = ano_fabricacao
        self.capacidade_carga_toneladas = capacidade_carga_toneladas

    def exibir_info(self):
        print(f"Veículo: {self.marca} {self.modelo} – Ano: {self.ano_fabricacao}")
```

Exceção Lançada
A capacidade de carga deve ser maior que 0

Analisar com o Copilot | Mostrar Pilha de Chamadas | Copiar Detalhes

Figura 24 Exceção lançada (erro de capacidade de carga)

```

class Carro(Veiculo):
    def __init__(self, marca: str, modelo: str, ano_fabricacao: int, numero_portas: int):
        super().__init__(marca, modelo, ano_fabricacao)
        if numero_portas not in [2, 4]:
            raise ValueError("A quantidade de portas deve ser 2 ou 4")
        self.numero_portas = numero_portas

    def exibir_info(self):
        super().exibir_info()
        print(f"Número de portas: {self.numero_portas}")

class Moto(Veiculo):
    def __init__(self, marca: str, modelo: str, ano_fabricacao: int, cilindrada: int):
        super().__init__(marca, modelo, ano_fabricacao)
        self.cilindrada = cilindrada

    def exibir_info(self):
        super().exibir_info()
        print(f"Cilindrada: {self.cilindrada}")

class Caminhao(Veiculo):
    def __init__(self, marca: str, modelo: str, ano_fabricacao: int, capacidade_carga_toneladas: int):
        super().__init__(marca, modelo, ano_fabricacao)
        self.capacidade_carga_toneladas = capacidade_carga_toneladas

    def exibir_info(self):
        super().exibir_info()
        print(f"Capacidade de carga: {self.capacidade_carga_toneladas} toneladas")

```

Exceção Lançada

A quantidade de portas deve ser 2 ou 4

Analisar com o Copilot
 Mostrar Pilha de Chamadas
 Copiar Detalhes

Figura 25 Exceção lançada (erro de quantidade de portas)

```

if not marca.strip():
    raise TypeError("A marca não pode ser vazia.")
self.marca = marca

if not modelo.strip():
    raise TypeError("O modelo não pode ser vazio.")
self.modelo = modelo

if ano_fabricacao not in range(1886, 2025):
    raise ValueError("O ano de fabricação deve estar entre 1886 e 2025.")
self.ano_fabricacao = ano_fabricacao

```

Exceção Lançada

A marca não pode ser vazia.

Analisar com o Copilot
 Mostrar Pilha de Chamadas
 Copiar Detalhes

Figura 26 Exceção lançada (falta de marca)

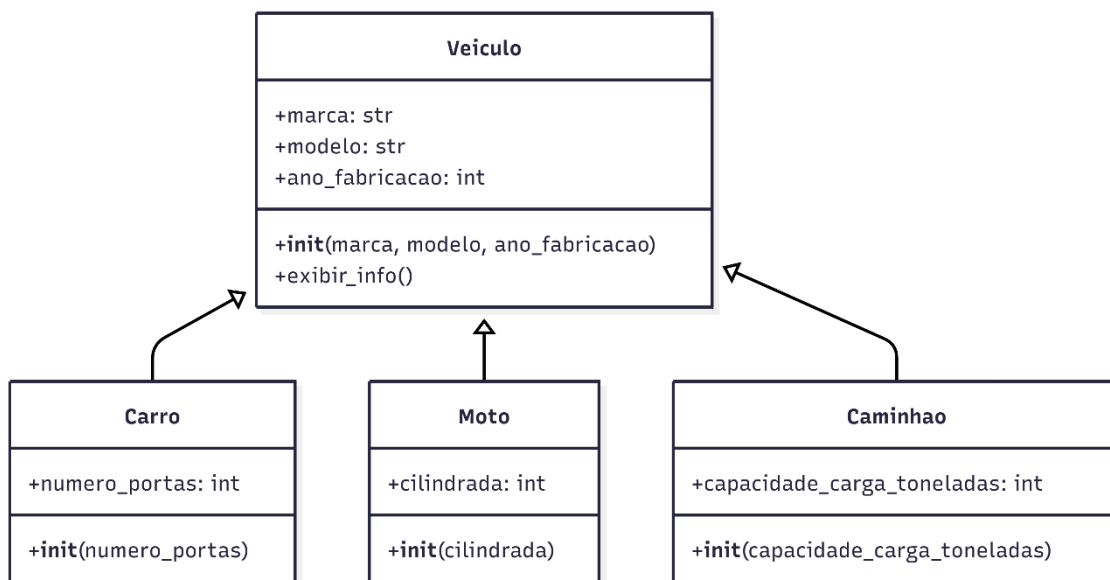


Figura 27 Diagrama UML

Exercício 2

Na herança simples, a classe filha herda apenas uma única classe mãe.

Exemplos:

- UsuarioComum (subclasse) herda de Usuario (superclasse);
- Moto (subclasse) herda de Veiculo (superclasse), Carro (subclasse) herda de Veiculo.

Na herança múltipla, uma classe filha herda de duas ou mais classes mães ao mesmo tempo.

Exemplo:

- Professor (subclasse) herda de Pessoa (superclasse) e Trabalhador (superclasse).

Exercício 3



```
pratica02_03.py
Comissionado
total_vendas

1 class Funcionario:
2     def __init__(self, nome: str, matricula: str):
3         if not nome:
4             raise ValueError("O nome do funcionário não pode ser vazio.")
5         self.nome = nome
6         if not matricula:
7             raise ValueError("A matrícula do funcionário não pode ser vazia.")
8         self.matricula = matricula
9
10    def calcular_pagamento(self):
11        print(f"Calculando pagamento para o funcionário {self.nome} (Matrícula: {self.matricula})")
12
13    class Assalariado(Funcionario):
14        def __init__(self, nome: str, matricula: str, salario: float):
15            super().__init__(nome, matricula)
16            if salario <= 0:
17                raise ValueError("O salário tem que ser > 0")
18            self.salario = salario
19
20        def calcular_pagamento(self):
21            super().calcular_pagamento()
22            return self.salario
23
24    class Horista(Funcionario):
25        def __init__(self, nome: str, matricula: str, horas_trabalhadas: int, valor_hora: float):
26            super().__init__(nome, matricula)
27            if horas_trabalhadas < 0:
28                raise ValueError("As horas trabalhadas têm que ser > 0.")
29            self.horas_trabalhadas = horas_trabalhadas
30            if valor_hora < 0:
31                raise ValueError("O valor da hora tem que ser >= 0.")
32            self.valor_hora = valor_hora
33
34        def calcular_pagamento(self):
35            super().calcular_pagamento()
36            return self.horas_trabalhadas * self.valor_hora
37
38    class Comissionado(Funcionario):
39        def __init__(self, salario_base: float, total_vendas: float, percentual_comissao: float, nome: str, matricula: str):
40            super().__init__(nome, matricula)
41            if salario_base <= 0:
42                raise ValueError("O salário base tem que ser > 0.")
43            self.salario_base = salario_base
44            if total_vendas < 0:
45                raise ValueError("O total de vendas tem que ser > 0.")
46            self.total_vendas = total_vendas
47            if percentual_comissao < 0 or percentual_comissao > 100:
48                raise ValueError("O percentual de comissão deve estar entre 0 e 100.")
49            self.percentual_comissao = percentual_comissao
50        def calcular_pagamento(self):
51            super().calcular_pagamento()
52            return self.salario_base + (self.total_vendas * self.percentual_comissao / 100)
53
54    class Financeiro:
55        def __init__(self, nome_departamento: str):
56            if not nome_departamento:
57                raise ValueError("O nome do departamento financeiro não pode ser vazio.")
58            self.nome_departamento = nome_departamento
59
60        def processar_pagamento(self, funcionario: Funcionario):
61            funcionario.calcular_pagamento()
62            print(
63                f"Departamento {self.nome_departamento} processou o pagamento de {funcionario.nome}, "
64                f"Matrícula: {funcionario.matricula} | Valor: R$ {funcionario.calcular_pagamento()}"
65            )
```

Figura 28 Módulo com classes e métodos

```

main.py* X prtica02_03.py
main
1  from prtica02_03 import *
2  def main():
3      financeiro1 = Financeiro("Departamento Financeiro")
4      assalariado1 = Assalariado("João", "555", 3000.00)
5      horista1 = Horista("Maria", "666", 160, 20.00)
6      comissionado1 = Comissionado(2000.00, 50000.00, 10, "Carlos", "777")
7
8      financeiro1.processar_pagamento(assalariado1)
9      financeiro1.processar_pagamento(horista1)
10     financeiro1.processar_pagamento(comissionado1)
11
12     main()

```

Figura 29 Main

```

C:\Users\Gui\AppData\Local\I X + v
Calculando pagamento para o funcionário João (Matrícula: 555)
Calculando pagamento para o funcionário João (Matrícula: 555)
Departamento Departamento Financeiro processou o pagamento de João, Matrícula: 555 | Valor: R$ 3000.0
Calculando pagamento para o funcionário Maria (Matrícula: 666)
Calculando pagamento para o funcionário Maria (Matrícula: 666)
Departamento Departamento Financeiro processou o pagamento de Maria, Matrícula: 666 | Valor: R$ 3200.0
Calculando pagamento para o funcionário Carlos (Matrícula: 777)
Calculando pagamento para o funcionário Carlos (Matrícula: 777)
Departamento Departamento Financeiro processou o pagamento de Carlos, Matrícula: 777 | Valor: R$ 7000.0
Press any key to continue . . . |

```

Figura 30 Saída no terminal

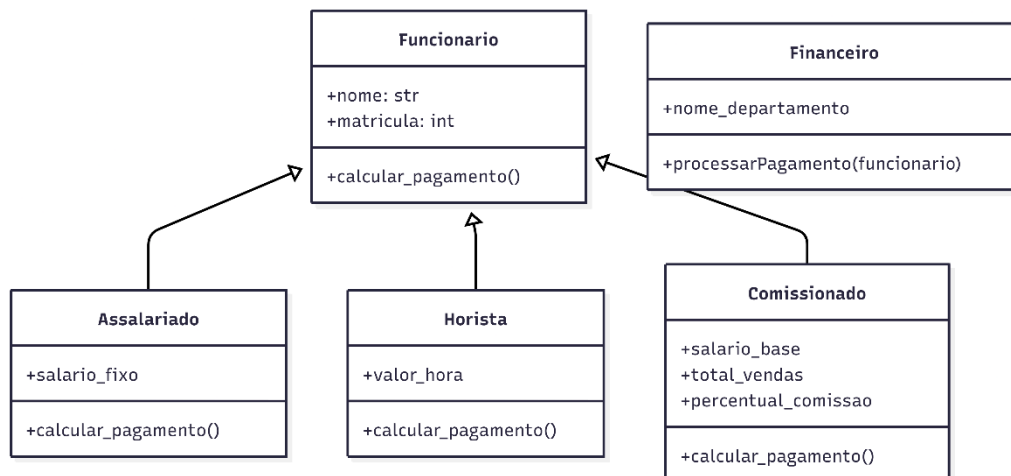


Figura 31 Diagrama UML

Atividade Prática 1 – parte 3 – APPOO

Exercício 1

```
1 class Ingresso:
2     def __init__(self, evento: str, valor):
3         if valor < 0:
4             raise ValueError("O valor do ingresso não pode ser negativo.")
5         if not evento:
6             raise ValueError("O nome do evento não pode ser vazio.")
7         self.evento = evento
8         self.valor = valor
9     def calcular_valor(self) -> float:
10         return self.valor
11
12 class IngressoVIP(Ingresso):
13     def __init__(self, evento: str, valor, valor_adicional):
14         super().__init__(evento, valor)
15         if valor_adicional < 0:
16             raise ValueError("O valor adicional do ingresso VIP não pode ser negativo.")
17         self.valor_adicional = valor_adicional
18     def calcular_valor(self) -> float:
19         return self.valor + self.valor_adicional
20
21 class IngressoMeiaEntrada(Ingresso):
22     def calcular_valor(self):
23         return super().calcular_valor() / 2
```

Figura 32 Módulo com classes e métodos

```
1 from pratica03_01 import *
2 def main():
3     evento = "Show de Rock"
4     valor = 100.00
5     taxa = 0.5 * valor
6
7     ingresso = Ingresso(evento, valor)
8     ingresso_vip = IngressoVIP(evento, valor, taxa)
9     ingresso_meia = IngressoMeiaEntrada(evento, valor)
10
11     ingressos = [ingresso, ingresso_vip, ingresso_meia]
12
13     for i in ingressos:
14         print(f"Tipo de ingresso: {type(i).__name__}")
15         print(f"Evento: {i.evento}")
16         print(f"Valor do ingresso: R$ {i.calcular_valor():.2f} \n")
17
18 if __name__ == "__main__":
19     main()
```

Figura 33 Main

```
C:\Users\Gui\AppData\Local\  X  +  v

Tipo de ingresso: Ingresso
Evento: Show de Rock
Valor do ingresso: R$ 100.00

Tipo de ingresso: IngressoVIP
Evento: Show de Rock
Valor do ingresso: R$ 150.00

Tipo de ingresso: IngressoMeiaEntrada
Evento: Show de Rock
Valor do ingresso: R$ 50.00

Press any key to continue . . . |
```

Figura 34 Saída no terminal

Exercício 2

```
from abc import ABC, abstractmethod

class Entrega(ABC):
    def __init__(self, codigo: str, distancia: float):
        if not codigo:
            raise ValueError("O código da entrega não pode ser vazio.")
        if distancia < 0:
            raise ValueError("A distância da entrega não pode ser negativa.")
        self.codigo = codigo
        self.distancia = distancia

    @abstractmethod
    def calcular_custo(self) -> float:
        pass

class EntregaMoto(Entrega):
    def __init__(self, codigo: str, distancia: float, valor_por_distancia: float):
        super().__init__(codigo, distancia)
        if valor_por_distancia < 0:
            raise ValueError("O valor por distância não pode ser negativo.")
        self.valor_por_distancia = valor_por_distancia

    def calcular_custo(self) -> float:
        return self.valor_por_distancia * self.distancia

class EntregaCaminhao(Entrega):
    def __init__(self, codigo: str, distancia: float, valor_por_distancia: float, taxa_adicional: float):
        super().__init__(codigo, distancia)
        if valor_por_distancia < 0:
            raise ValueError("O valor por distância não pode ser negativo.")
        if taxa_adicional < 0:
            raise ValueError("A taxa adicional não pode ser negativa.")
        self.valor_por_distancia = valor_por_distancia
        self.taxa_adicional = taxa_adicional

    def calcular_custo(self) -> float:
        return (self.valor_por_distancia * self.distancia) + self.taxa_adicional
```

Figura 35 Módulo com classes e métodos

```
main.py* ✕
main
1  from pratica03_02 import *
2
3  def main():
4      entrega_moto = EntregaMoto("A123", 4.7, 3.0)
5      entrega_caminhao = EntregaCaminhao("B456", 12.0, 9.0, 40.0)
6      #entrega_erro = Entrega("C789", 5.0)
7
8      print(f"Custo da entrega de moto: {entrega_moto.calcular_custo():.2f}")
9      print(f"Custo da entrega de caminhão: {entrega_caminhao.calcular_custo():.2f}")
10
11  main()
```

Figura 36 Main

```
main.py ✕
main
1  from pratica03_02 import *
2
3  def main():
4      entrega_moto = EntregaMoto("A123", 4.7, 3.0)
5      entrega_caminhao = EntregaCaminhao("B456", 12.0, 9.0, 40.0)
6      entrega_erro = Entrega("C789", 5.0)
7
8      print(f"Custo da entrega de moto: {e
9      print(f"Custo da entrega de caminhão
10
11  main()
```

Exceção Lançada
Can't instantiate abstract class Entrega without an implementation for abstract method 'calcular_custo'

Analisar com o Copilot Mostrar Pilha de Chamadas Copiar Detalhes

Figura 37 Erro causado ao tentar criar uma entrega genérica

```
C:\Users\Gui\AppData\Local\I ✕ + v
Custo da entrega de moto: 14.10
Custo da entrega de caminhão: 148.00
Press any key to continue . . . |
```

Figura 38 Saída no terminal